RAPPORT TECHNIQUE

CLOUD · INFRASTRUCTURE AS CODE · AWS

Infrastructure

Cloud d’une PME

AUTEUR

Saïd SANGARE

DATE

Octobre 2026

STACK

AWS · Terraform · Linux · Windows

SOMMAIRE

Table des matières

01 Résumé exécutif
02 Contexte, objectifs et périmètre
03 Architecture technique et topologie réseau
04 Environnement et prérequis
05 Implémentation Terraform
06 Déploiement
07 Validation et tests de fonctionnement
08 Analyse, limites et perspectives
09 Nettoyage des ressources
10 Synthèse des compétences validées

01 Résumé Exécutif

Dans la majorité des portfolios de cybersécurité, les projets de détection ou de durcissement sont présentés de manière isolée, sans l'infrastructure d'entreprise réaliste sur laquelle ils s'appliqueraient. En pratique, l'ordre logique est inverse : une infrastructure existe d'abord, souvent construite sans vision sécurité dès l'origine, puis un travail de sécurisation vient la corriger et la renforcer.

Le présent rapport documente la conception, le déploiement et la validation de l'équivalent cloud du réseau d'une petite ou moyenne entreprise : trois segments réseau (DMZ, Serveurs, Postes utilisateurs), un serveur Windows combinant Active Directory et partage de fichiers, un serveur applicatif interne, trois postes de travail, un site web public et un DNS interne. L'ensemble est entièrement décrit en code avec **Terraform** et déployé sur **AWS** (région `eu-west-1`, Irlande).

Le déploiement, reproductible et versionnable, crée environ 25 ressources, dont 7 instances EC2 (1 Windows Server et 6 Linux). Les tests de validation démontrent la cohérence de la segmentation : accès Internet sortant des zones privées via une instance NAT, résolution DNS interne via Route 53, accès à l'intranet depuis les postes, partage de fichiers sur le serveur Windows et site vitrine joignable depuis Internet.

Volontairement non durcie à ce stade, cette infrastructure constitue la base sur laquelle un second projet (approche Zero Trust) viendra appliquer des mesures de sécurité, permettant de documenter une progression « avant / après ».

3	7	≈ 25	100 %
segments réseau : DMZ, Serveurs, Postes	instances EC2 : 1 Windows + 6 Linux	ressources créées par Terraform	du déploiement décrit en code

02 Contexte, Objectifs et Périmètre

2.1. Objectifs

- Concevoir une architecture réseau segmentée réaliste (DMZ / Serveurs / Postes) et non une machine isolée.
- Transposer les éléments d'un réseau physique de PME vers des ressources cloud AWS équivalentes.
- Automatiser l'intégralité du déploiement avec Terraform (Infrastructure as Code) pour garantir reproductibilité et traçabilité.
- Valider le fonctionnement par une série de contrôles ordonnés, appuyés par des captures d'écran.

2.2. Transposition d'une PME physique vers le cloud

Le tableau suivant présente la correspondance retenue entre les composants d'un réseau d'entreprise classique et leurs équivalents AWS.

Élément physique d'une PME	Équivalent AWS utilisé	Rôle
Switch / VLAN (segmentation)	Sous-réseaux (subnets) séparés	Isoler serveurs, postes et zone publique
Routeur / Box Internet	Instance NAT (petite EC2)	Accès Internet sortant partagé et contrôlé
Pare-feu	Security Groups	Filtrer qui peut parler à qui, sur quel port
Contrôleur de domaine (Active Directory)	Windows Server EC2 avec rôle AD DS	Authentification centralisée (simulée)
Serveur de fichiers	Même Windows Server, rôle Fichiers	Partage centralisé de documents
Serveur applicatif interne (intranet)	Instance Linux EC2	Application métier accessible en interne
Postes de travail (Comptabilité, RH, Direction)	3 instances Linux EC2	Simulent les utilisateurs
Site web public	Instance Linux EC2 en DMZ	Vitrine accessible depuis Internet
Serveur DNS interne	Route 53 Private Hosted Zone	Résolution de noms internes (ex. `fichiers.pme.local`)

2.3. Périmètre : ce que l'infrastructure n'est volontairement pas (encore)

Cette version est volontairement non sécurisée au sens strict : les postes peuvent communiquer avec les serveurs sans restriction fine, il n'y a ni chiffrement dédié, ni journalisation avancée, ni isolation totale. Ce choix est assumé : le projet suivant (Zero Trust) comblera ces manques sur cette même base.

03 Architecture Technique et Topologie Réseau

L'infrastructure repose sur un VPC unique (`10.0.0.0/16`) découpé en trois sous-réseaux situés dans la zone de disponibilité `eu-west-1a`. L'Internet Gateway n'est directement utilisée que par la DMZ ; les deux zones privées sortent vers Internet exclusivement par l'instance NAT.

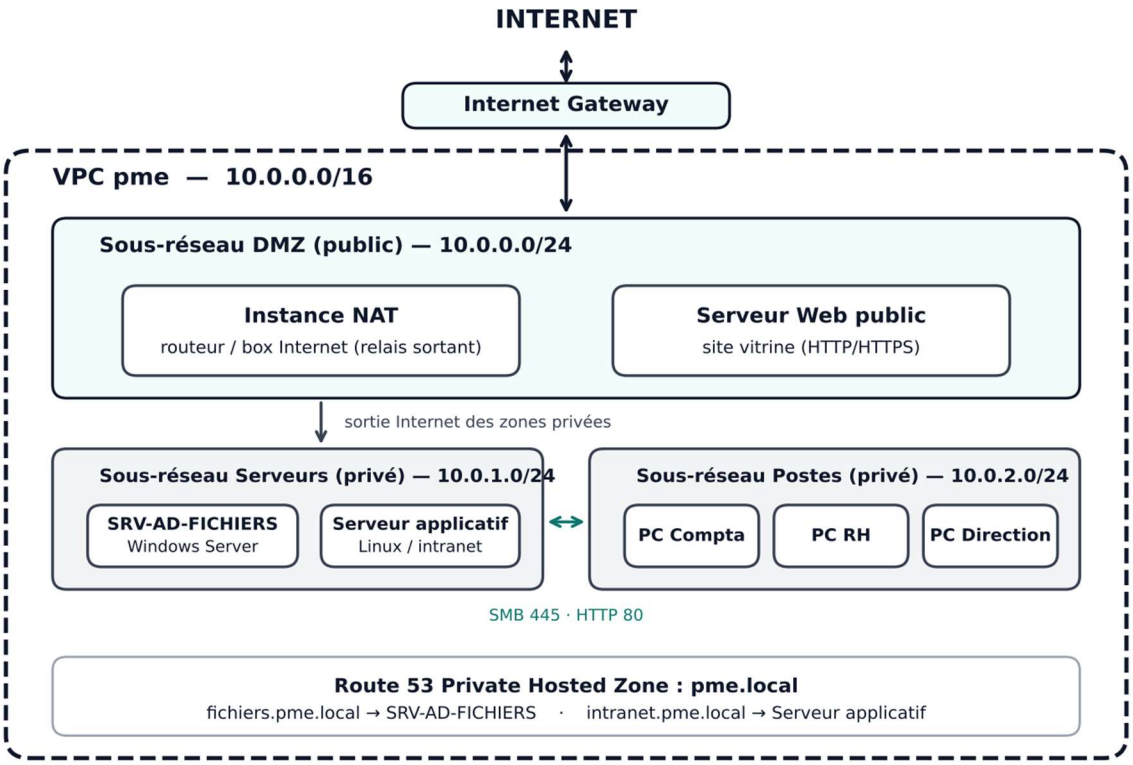


Schéma 1 : Architecture globale du VPC (DMZ, Serveurs, Postes, DNS interne).

3.1. Plan d'adressage

Sous-réseau	Plage CIDR	Zone de disponibilité	Type	Contenu
DMZ	10.0.0.0/24	eu-west-1a	Public	Instance NAT, serveur web public
Serveurs	10.0.1.0/24	eu-west-1a	Privé	Serveur AD/Fichiers, serveur applicatif
Postes utilisateurs	10.0.2.0/24	eu-west-1a	Privé	3 postes (Compta, RH, Direction)

3.2. Flux réseau prévus

- Les **postes utilisateurs** accèdent au **serveur de fichiers** (SMB, port 445) et au **serveur applicatif** (HTTP, port 80).
- Les **serveurs et postes privés** accèdent à Internet en sortie uniquement via l'**instance NAT** (aucune IP publique directe).

- Le **serveur web public** est accessible depuis Internet en HTTP/HTTPS uniquement.
- La résolution de noms internes (`fichiers.pme.local`, `intranet.pme.local`) passe par la **zone DNS privée Route 53**.

04 Environnement et Prérequis

Le principe général est le suivant : Terraform ne sait pas communiquer seul avec AWS ; il s'appuie sur les identifiants stockés localement par l'AWS CLI pour s'authentifier à chaque commande.

4.1. Outils installés

Outil	Rôle	Vérification
Git	Historique et sauvegarde du code	`git --version`
AWS CLI v2	Communication avec le compte AWS depuis un terminal	`aws --version`
Terraform	Écriture et exécution du code d'infrastructure	`terraform --version`
Visual Studio Code	Éditeur des fichiers `.tf` (extension HashiCorp Terraform)	`code --version`

Terraform n'ayant pas d'installeur, `terraform.exe` est placé dans un dossier dédié (`C:\terraform`) ajouté à la variable d'environnement `Path`, afin d'être accessible depuis n'importe quel terminal.

4.2. Compte AWS, identifiants et budget

- **Utilisateur IAM dédié** `terraform-admin` (politique `AdministratorAccess`) : le compte root n'est jamais utilisé au quotidien. Ce niveau de droits est une simplification de laboratoire ; en entreprise, le principe du moindre privilège s'applique.
- **Clés d'accès** générées pour la CLI et configurées via `aws configure` (région `eu-west-1`, sortie `json`). Elles ne sont jamais publiées sur GitHub.
- **Budget de sécurité** (service AWS Budgets) avec alertes e-mail à 50 %, 80 % et 100 % du montant défini.

```
aws configure
aws sts get-caller-identity   # vérifie la connexion Terraform/AWS
```

4.3. Notions Terraform utilisées

Notion / commande	Description
`provider`	Plugin permettant à Terraform de dialoguer avec l'API d'AWS.
`resource`	Élément d'infrastructure à créer (type + nom choisi).

Notion / commande	Description
`terraform.tfstate`	Fichier d'état : mémoire de Terraform sur ce qu'il a créé. Ne jamais le supprimer manuellement.
`terraform init`	Télécharge les providers nécessaires (une fois par projet).
`terraform plan`	Calcule et affiche les changements prévus, sans rien créer.
`terraform apply`	Exécute les changements après confirmation.
`terraform destroy`	Supprime toutes les ressources créées par le projet.

05 Implémentation Terraform

Le code est organisé en fichiers thématiques, chacun correspondant à une brique de l'infrastructure :

```
projet1-pme/
|-- providers.tf          (provider AWS)
|-- variables.tf          (variables du projet)
|-- terraform.tfvars      (valeurs concrètes)
|-- locals.tf             (tags communs)
|-- network.tf            (VPC, sous-réseaux, routage, Internet Gateway)
|-- nat-instance.tf       (instance NAT, "box internet")
|-- security.tf           (Security Groups par segment)
|-- iam.tf                (rôle et profil SSM pour l'administration)
|-- serveur-ad-fichiers.tf (Windows Server : AD + fichiers)
|-- serveur-applicatif.tf (serveur intranet Linux)
|-- postes-utilisateurs.tf (les 3 "PC" Linux)
|-- serveur-web-public.tf (site vitrine, en DMZ)
|-- dns-interne.tf        (Route 53 Private Hosted Zone)
`-- outputs.tf            (informations affichées après déploiement)
```

> **À RETENIR** **Attention coût.** Le projet crée 7 instances EC2 dont un Windows Server. Une infrastructure oubliée plusieurs jours peut représenter un coût réel : l'exécution finale de `terraform destroy` est indispensable.

5.1. Provider et variables (providers.tf, variables.tf, locals.tf)

Le provider AWS est épinglé en version `~> 5.0` et configuré sur la région `eu-west-1`. L'état Terraform reste **local** : la gestion d'état distant est volontairement reportée au projet suivant.

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" {
  region = "eu-west-1"
}
```

Les variables centralisent les paramètres : région, nom de l'entreprise (`pme`), plages CIDR du VPC et des trois sous-réseaux, taille des instances (`t3.micro`) et mot de passe administrateur Windows. Ce dernier est déclaré `sensitive = true` afin qu'il n'apparaisse pas en clair dans les sorties de `plan` et `apply`, et le fichier `terraform.tfvars` est exclu du dépôt Git via `.gitignore`.

```
variable "mot_de_passe_admin_windows" {
  description = "Mot de passe administrateur du Windows Server"
  type        = string
  sensitive   = true
}

locals {
  tags_communs = {
    Entreprise = var.nom_entreprise
    ManagedBy  = "terraform"
    Projet     = "infrastructure-pme"
  }
}
```

Les tags communs, fusionnés à chaque ressource avec `merge()`, garantissent un étiquetage homogène (entreprise, outil de gestion, projet) facilitant le suivi des coûts et l'inventaire.

5.2. Réseau et segmentation (network.tf)

Le fichier définit le VPC (avec résolution DNS activée), l'Internet Gateway, les trois sous-réseaux et deux tables de routage :

- une **table publique**, dont la route par défaut (`0.0.0.0/0`) pointe vers l'Internet Gateway, associée à la DMZ ;
- une **table privée**, associée aux sous-réseaux Serveurs et Postes, dont la route vers Internet est ajoutée dans `nat-instance.tf` et passe par l'instance NAT.

```
resource "aws_vpc" "reseau_pme" {
  cidr_block           = var.cidr_vpc
  enable_dns_support   = true
  enable_dns_hostnames = true
}

resource "aws_subnet" "dmz" {
  vpc_id                  = aws_vpc.reseau_pme.id
  cidr_block              = var.cidr_dmz
  availability_zone       = "${var.region}a"
  map_public_ip_on_launch = true
}

resource "aws_route_table" "table_publique" {
  vpc_id = aws_vpc.reseau_pme.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.porte_internet.id
  }
}
```

> **POINT CLÉ** C'est cette logique de routage qui différencie un sous-réseau « public » d'un sous-réseau « privé » sur AWS : la DMZ route directement vers l'Internet Gateway, tandis que Serveurs et Postes n'ont aucune route directe vers Internet.

5.3. Instance NAT (nat-instance.tf)

AWS propose un service managé « NAT Gateway », facturé en continu dès sa création (environ 0,045 $/heure, soit une trentaine de dollars par mois). Pour respecter un budget de petite structure, une **instance NAT** (petite EC2 Amazon Linux 2023 jouant le rôle de routeur) a été retenue. Ce choix est moins résilient (si l'instance tombe, plus aucun accès Internet sortant) mais nettement moins coûteux : un compromis réel de PME.

```
resource "aws_instance" "nat" {
  ami                         = data.aws_ami.amazon_linux_nat.id
  instance_type               = var.type_instance_linux
  subnet_id                   = aws_subnet.dmz.id
  vpc_security_group_ids      = [aws_security_group.sg_nat.id]
  source_dest_check           = false   # indispensable pour relayer le trafic
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    sysctl -w net.ipv4.ip_forward=1
    echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.conf
    iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
    service iptables save
  EOF
}

resource "aws_route" "route_vers_nat" {
  route_table_id         = aws_route_table.table_privee.id
  destination_cidr_block = "0.0.0.0/0"
  network_interface_id   = aws_instance.nat.primary_network_interface_id
}
```

> POINT CLÉ **Point technique clé : source_dest_check = false.** Par défaut, AWS bloque tout trafic dont l'instance n'est ni source ni destination. Or une instance NAT doit justement relayer le trafic d'autres machines : sans cette désactivation, le NAT ne fonctionnerait jamais, même avec une configuration système parfaite. Le script `user_data` active le relayage IP (`ip_forward`) et le masquage d'adresses (`MASQUERADE`) au démarrage.

5.4. Security Groups par segment (security.tf)

Chaque Security Group joue le rôle d'une règle de pare-feu : il définit qui peut parler à qui, et sur quel port.

Security Group	Flux entrants autorisés	Source
`sg_nat`	Tout trafic	Plage du VPC (`10.0.0.0/16`)
`sg_serveurs`	SMB 445, HTTP 80, RDP 3389	Plage des postes (`10.0.2.0/24`) uniquement
`sg_postes`	SSH 22 (administration / démonstration)	Plage du VPC
`sg_web_public`	HTTP 80, HTTPS 443	Internet (`0.0.0.0/0`)

Même à ce stade « non durci », les serveurs n'acceptent que les ports 445, 80 et 3389 et uniquement depuis la plage des postes utilisateurs, et non depuis n'importe où : une segmentation simple mais cohérente avec une PME correctement administrée.

5.5. Administration via AWS Systems Manager (iam.tf)

Les instances n'exposent aucun accès d'administration direct depuis Internet. Un **rôle IAM** (politique gérée `AmazonSSMManagedInstanceCore`) et un **profil d'instance** associé (`profil_ssm`) sont attachés à chaque instance via l'attribut `iam_instance_profile`. L'agent SSM, qui joint les services AWS grâce à la sortie Internet fournie par l'instance NAT, permet alors d'ouvrir des sessions shell sur les instances Linux (Session Manager) et une session Bureau à distance sur le Windows Server (Fleet Manager).

```
resource "aws_instance" "serveur_web_public" {
  # ...
  iam_instance_profile = aws_iam_instance_profile.profil_ssm.name
}
```

5.6. Serveur Windows : Active Directory et fichiers (serveur-ad-fichiers.tf)

Cette brique est la plus typique d'une PME : un serveur Windows combinant l'annuaire Active Directory et le partage de fichiers. Il est déployé à partir de l'AMI Windows Server 2022 dans le sous-réseau Serveurs. Les instances Windows attendent un script **PowerShell** encadré par les balises `<powershell>` (équivalent de `cloud-init` pour Linux). Le script :

1. renomme la machine en `SRV-AD-FICHIERS` ;
2. définit le mot de passe administrateur fourni via Terraform ;
3. installe le rôle **AD-Domain-Services** ;
4. installe le rôle **Serveur de fichiers** et crée le dossier de partage `C:\PartagePME` ;
5. redémarre la machine pour finaliser l'installation.

```
user_data = <<-EOF
  <powershell>
  Rename-Computer -NewName "SRV-AD-FICHIERS" -Force
  $motdepasse = ConvertTo-SecureString "${var.mot_de_passe_admin_windows}" -AsPlainText -Force
  Set-LocalUser -Name "Administrator" -Password $motdepasse
  Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools
  Install-WindowsFeature -Name FS-FileServer -IncludeManagementTools
  New-Item -Path "C:\PartagePME" -ItemType Directory -Force
  Restart-Computer -Force
  </powershell>
EOF
```

> **À RETENIR** Le script installe les rôles Windows mais ne réalise pas la promotion complète en contrôleur de domaine (`dcpromo` / `Install-ADDSForest`), qui nécessite une configuration plus poussée (nom de domaine, niveau fonctionnel). Ce périmètre suffit à démontrer la compétence ; la promotion constitue une piste d'évolution.

5.7. Serveur applicatif, postes utilisateurs et serveur web public

- **Serveur applicatif** (`serveur-applicatif.tf`) : instance Linux dans le sous-réseau Serveurs, installant Apache (`httpd`) et publiant une page « Intranet de la PME ».
- **Postes utilisateurs** (`postes-utilisateurs.tf`) : trois instances Linux (comptabilité, RH, direction) dans le sous-réseau Postes, avec l'outil `cifs-utils` pour accéder au partage SMB.
- **Serveur web public** (`serveur-web-public.tf`) : instance Linux en DMZ avec IP publique, publiant la page « Bienvenue chez notre PME ».

Les trois postes sont créés par **une seule** ressource grâce à la boucle `for_each`, qui répète la structure pour chaque élément de la liste `local.departements` ; `each.key` personnalise le nom de chaque instance.

```
locals {
  departements = ["comptabilite", "rh", "direction"]
}

resource "aws_instance" "postes" {
  for_each = toset(local.departements)
  ami                    = data.aws_ami.amazon_linux_poste.id
  instance_type          = var.type_instance_linux
  subnet_id              = aws_subnet.postes.id
  vpc_security_group_ids = [aws_security_group.sg_postes.id]
  tags = merge(local.tags_communs, {
    Name         = "${var.nom_entreprise}-pc-${each.key}"
    Departement  = each.key
  })
}
```

> **POINT CLÉ** Ajouter un poste « Marketing » ne demande que d'ajouter `"marketing"` à la liste `local.departements` : Terraform créera la nouvelle instance au prochain `apply`, sans toucher au reste du code. C'est l'intérêt de l'Infrastructure as Code face à une création manuelle.

5.8. DNS interne et sorties (dns-interne.tf, outputs.tf)

Sans DNS interne, chaque poste devrait connaître par cœur l'adresse IP des serveurs. Une **Private Hosted Zone Route 53** (`pme.local`), rattachée au VPC, expose des noms lisibles : `fichiers.pme.local` (serveur AD/Fichiers) et `intranet.pme.local` (serveur applicatif), via des enregistrements de type A (TTL 300 s).

```
resource "aws_route53_zone" "zone_interne" {
  name = "pme.local"
  vpc { vpc_id = aws_vpc.reseau_pme.id }
}

resource "aws_route53_record" "intranet" {
  zone_id = aws_route53_zone.zone_interne.zone_id
  name    = "intranet.pme.local"
  type    = "A"
  ttl     = 300
  records = [aws_instance.serveur_applicatif.private_ip]
}
```

Le fichier `outputs.tf` expose à l'issue du déploiement l'IP publique du site web (`ip_publique_site_web`), l'IP publique du NAT, l'IP privée du serveur de fichiers et les identifiants des postes utilisateurs.

06 Déploiement

Le déploiement suit le cycle standard Terraform : initialisation, planification, application.

6.1. Initialisation et planification

```
cd projet1-pme
terraform init
terraform plan
```

La commande `terraform plan` calcule les changements sans rien créer. Elle annonce environ 25 ressources prévues : VPC, 3 sous-réseaux, tables de routage, 7 instances, Security Groups, zone DNS, etc.

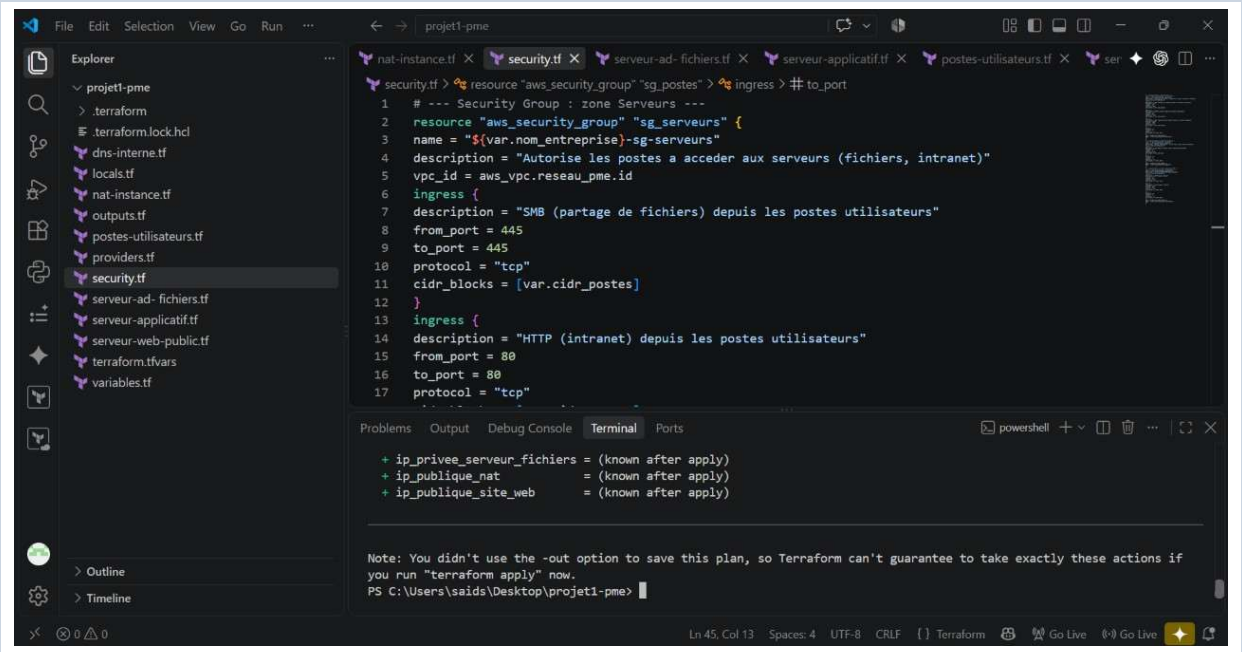


Figure 1 *Fin de la sortie de terraform plan : ressources prévues et valeurs de sortie calculées après déploiement.*

6.2. Application

```
terraform apply   # confirmer avec "yes"
```

Le Windows Server est plus long à démarrer que les instances Linux (souvent 5 à 10 minutes, redémarrage inclus).

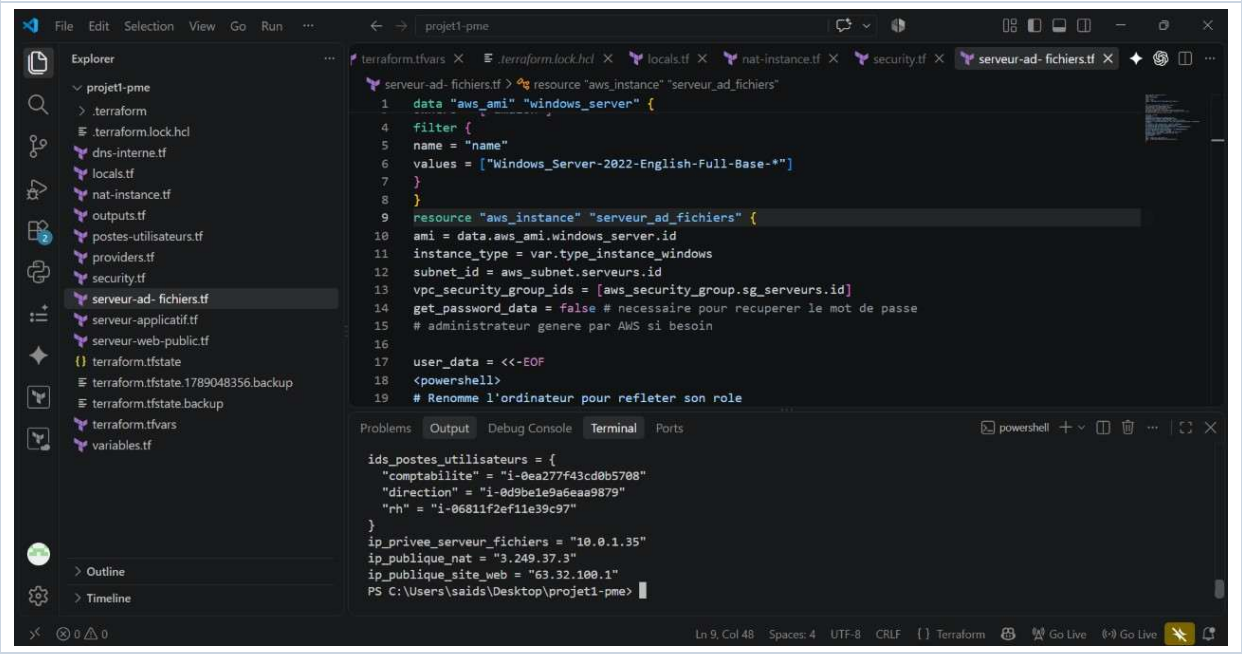


Figure 2 *Fin du déploiement terraform apply : valeurs de sortie (identifiants des postes, IP privée du serveur de fichiers, IP publiques du NAT et du site web).*

07 Validation et Tests de Fonctionnement

Les contrôles suivants sont exécutés dans un ordre précis : d'abord l'inventaire des ressources créées (réseau, instances), puis les flux et services depuis l'extérieur et depuis l'intérieur du réseau, et enfin le serveur Windows.

7.1. Vue d'ensemble du réseau créé

Dans la console AWS (service VPC, menu « Subnets », filtre sur le VPC du projet), les trois sous-réseaux doivent apparaître avec leurs plages CIDR : DMZ (`10.0.0.0/24`), Serveurs (`10.0.1.0/24`) et Postes utilisateurs (`10.0.2.0/24`).

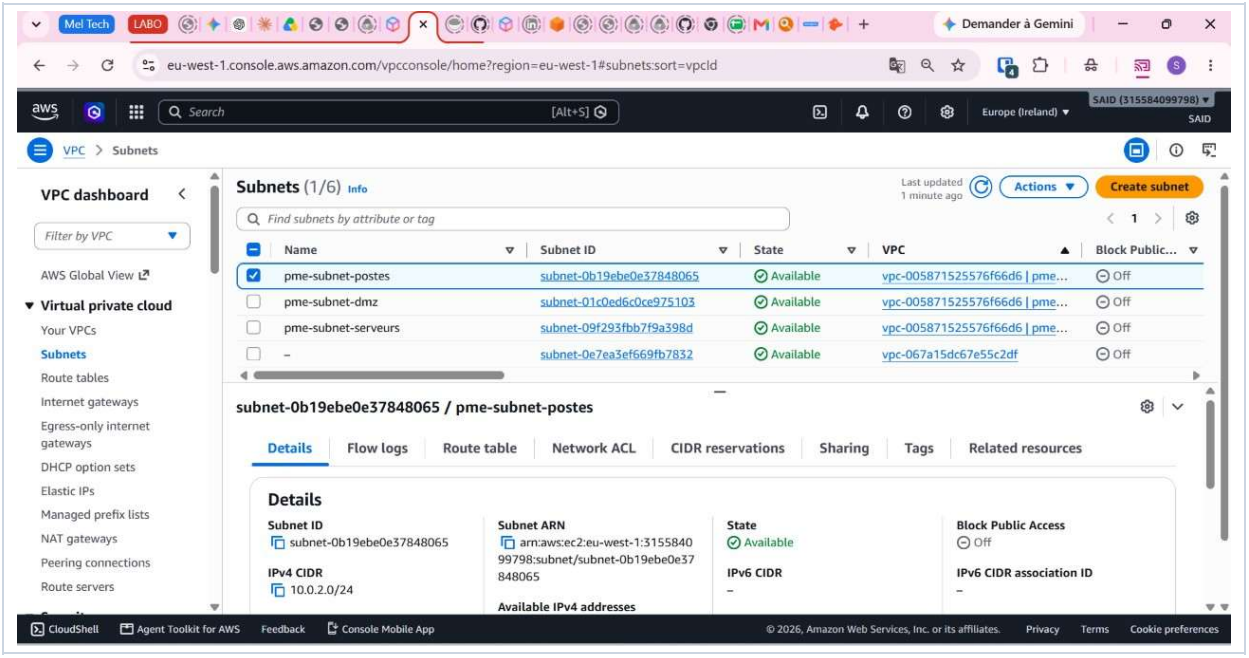


Figure 3 *Console AWS VPC — liste des 3 sous-réseaux (DMZ, Serveurs, Postes Utilisateurs) avec leurs plages CIDR.*

7.2. Vue d'ensemble des instances

Dans la console AWS (service EC2, menu « Instances »), les 7 instances doivent être à l'état « running » : NAT, serveur web public, serveur applicatif, serveur AD/Fichiers et les 3 postes.

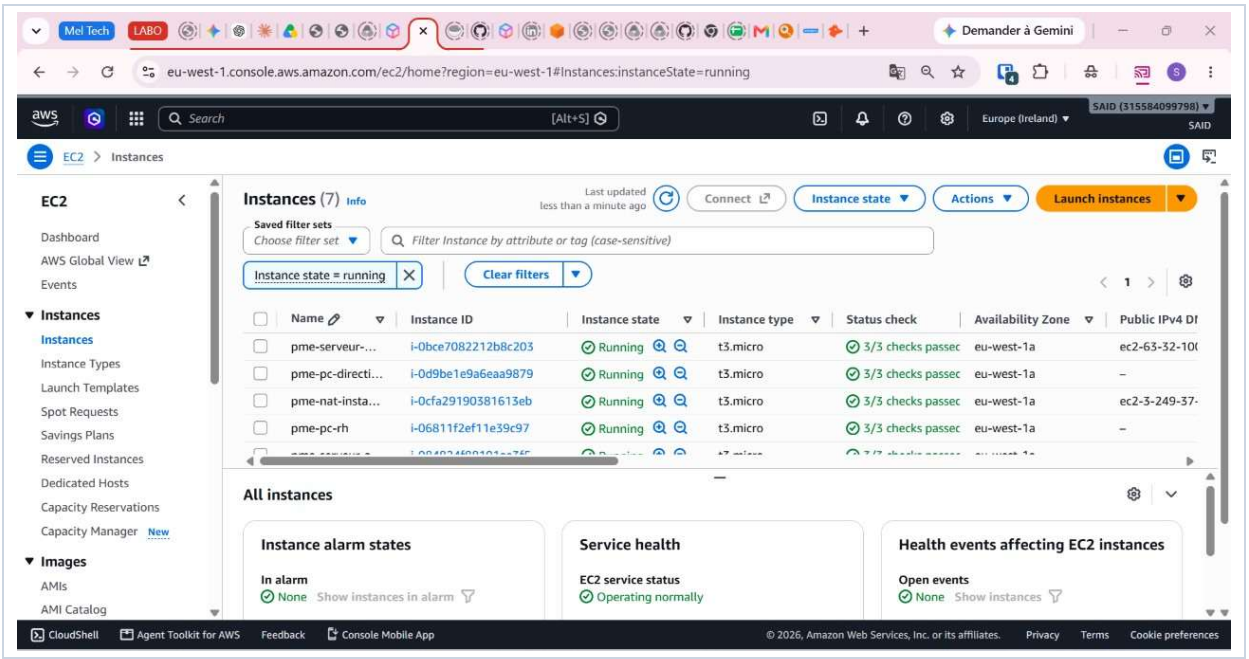


Figure 4 *Console EC2 — les 7 instances de l'infrastructure à l'état « running ».*

7.3. Le site web public

L'adresse IP publique obtenue en sortie de `terraform apply` (`ip_publique_site_web`) est ouverte dans un navigateur :

```
http://<IP_PUBLIQUE>
```

La page « Bienvenue chez notre PME » doit s'afficher, confirmant la publication du site vitrine depuis la DMZ.

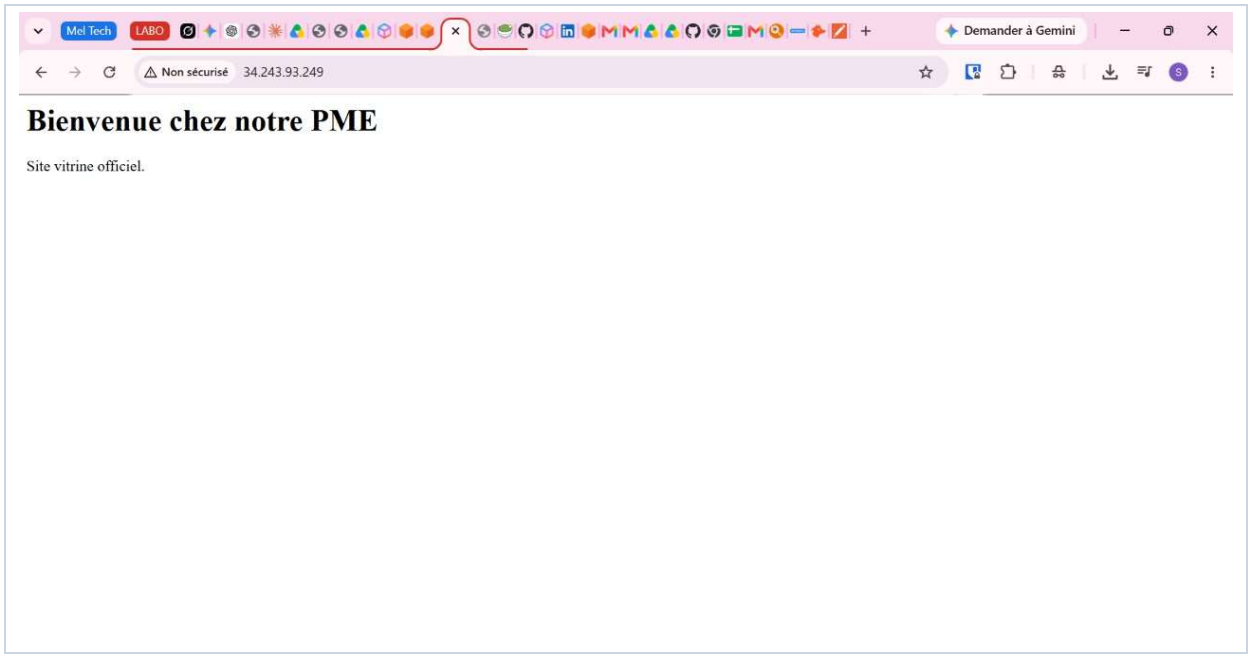


Figure 5 *Site vitrine de la PME accessible depuis Internet.*

7.4. Accès Internet sortant depuis une zone privée

Une session est ouverte sur un poste utilisateur (ici `pme-pc-rh`) via AWS Systems Manager (Session Manager) :

```
aws ssm start-session --target <ID_POSTE_RH>
```

Depuis cette session, un test de sortie vers Internet est réalisé :

```
curl -I https://www.google.com
```

Une réponse `HTTP/2 200` prouve que le routage via l'instance NAT est opérationnel : le poste, dépourvu d'adresse IP publique, accède à Internet par la DMZ.

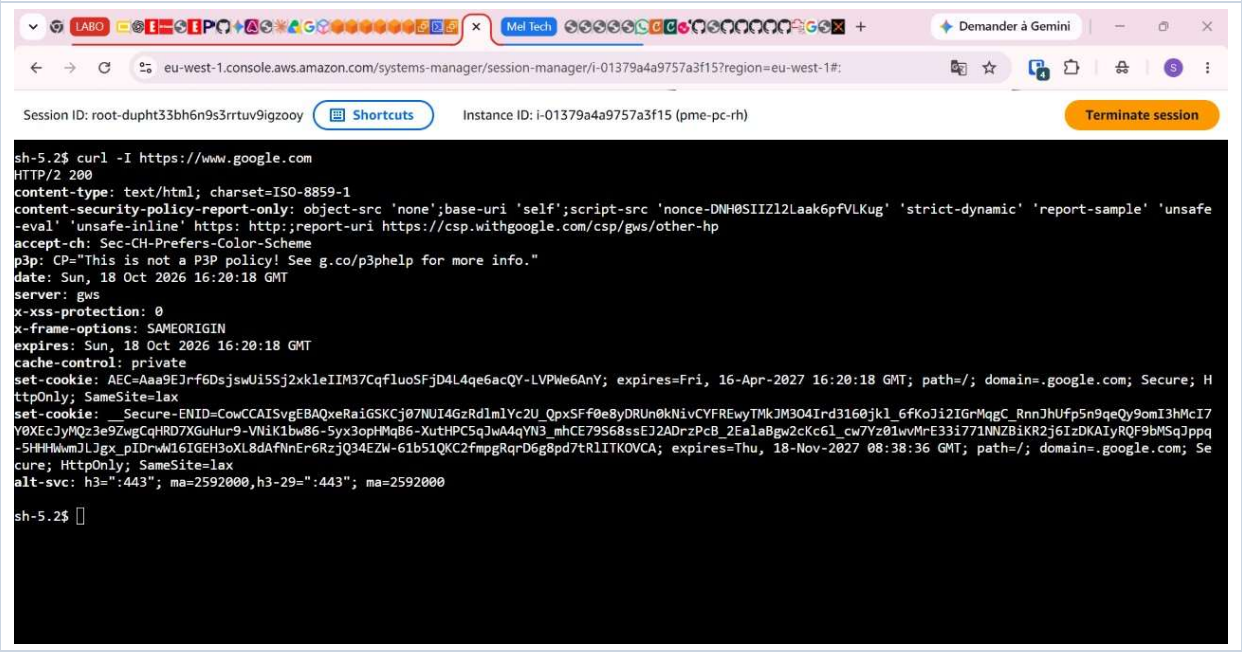


Figure 6 *Test curl -I vers Internet depuis un poste privé : réponse HTTP/2 200, preuve du fonctionnement de l'instance NAT.*

7.5. Résolution DNS interne

Toujours dans la même session (poste utilisateur), la résolution des noms internes est testée :

```
nslookup fichiers.pme.local
nslookup intranet.pme.local
```

Les deux commandes doivent renvoyer les adresses IP privées correspondantes.

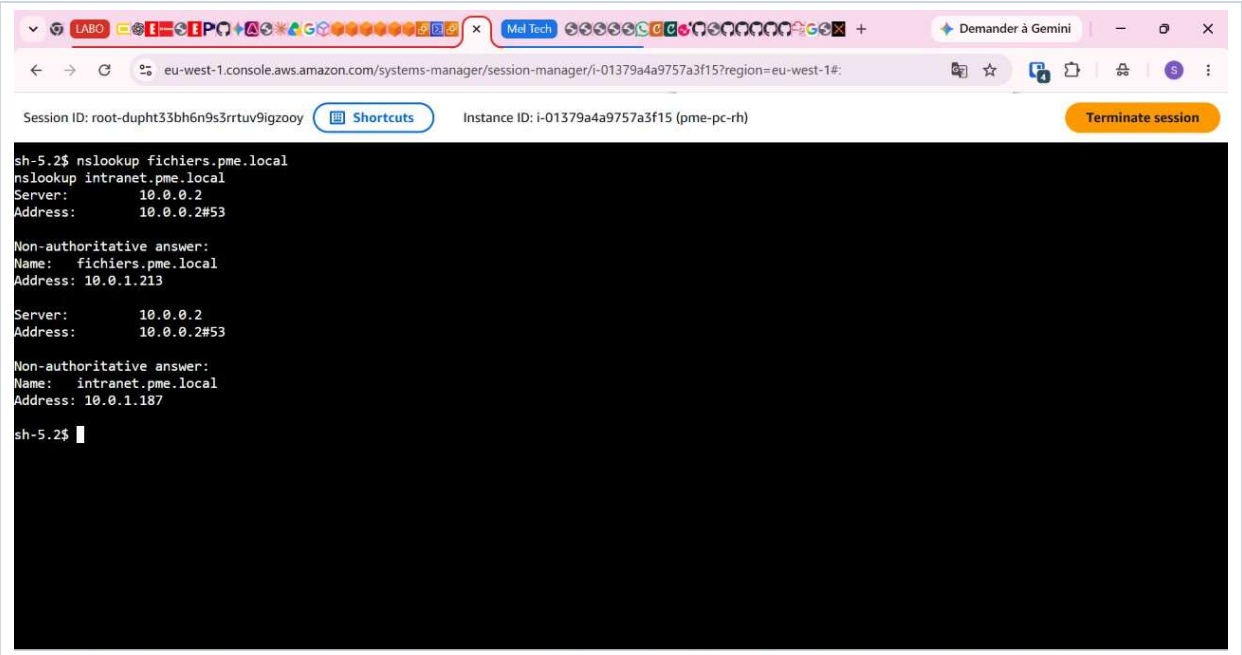


Figure 7 *Résultats des commandes nslookup pour fichiers.pme.local et intranet.pme.local.*

Les enregistrements correspondants sont visibles dans la console Route 53, menu « Hosted zones », zone `pme.local`.

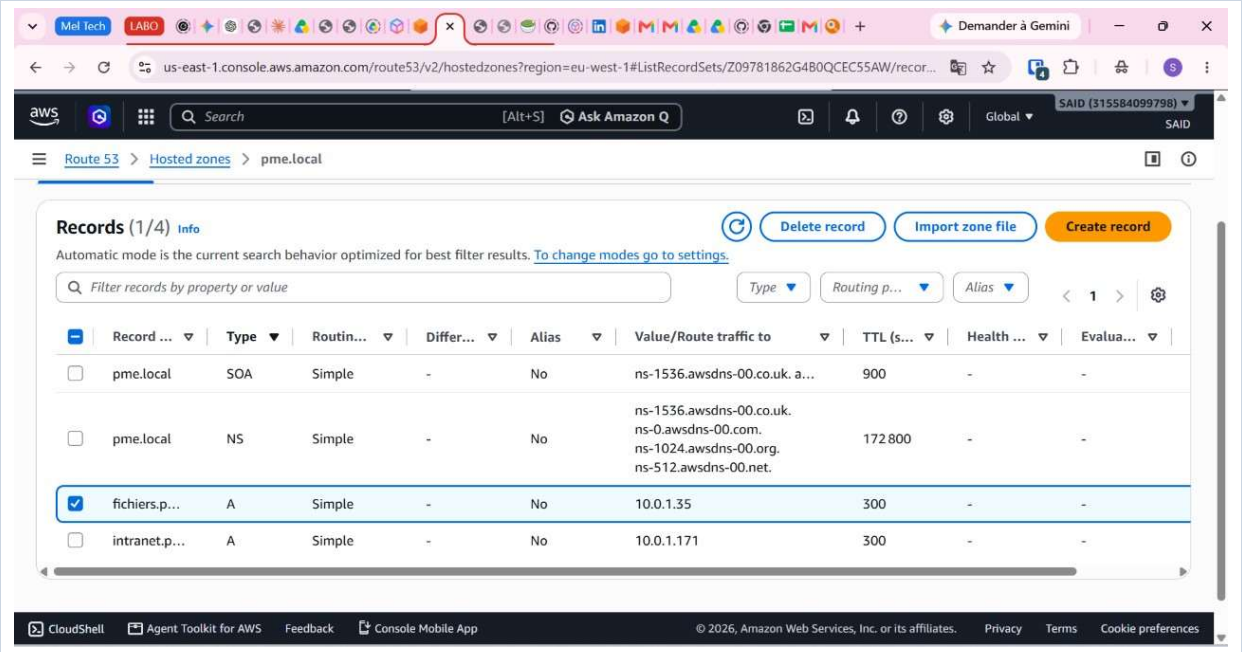


Figure 8 *Console Route 53 — enregistrements DNS de la zone privée pme.local.*

7.6. Accès au serveur applicatif interne

Toujours depuis la même session, le serveur intranet est interrogé par son nom DNS interne :

```
curl http://intranet.pme.local
```

La commande doit renvoyer le contenu HTML de la page « Intranet de la PME », prouvant à la fois l'autorisation du flux HTTP depuis la zone Postes et la résolution DNS interne. La session est ensuite fermée avec la commande `exit`.

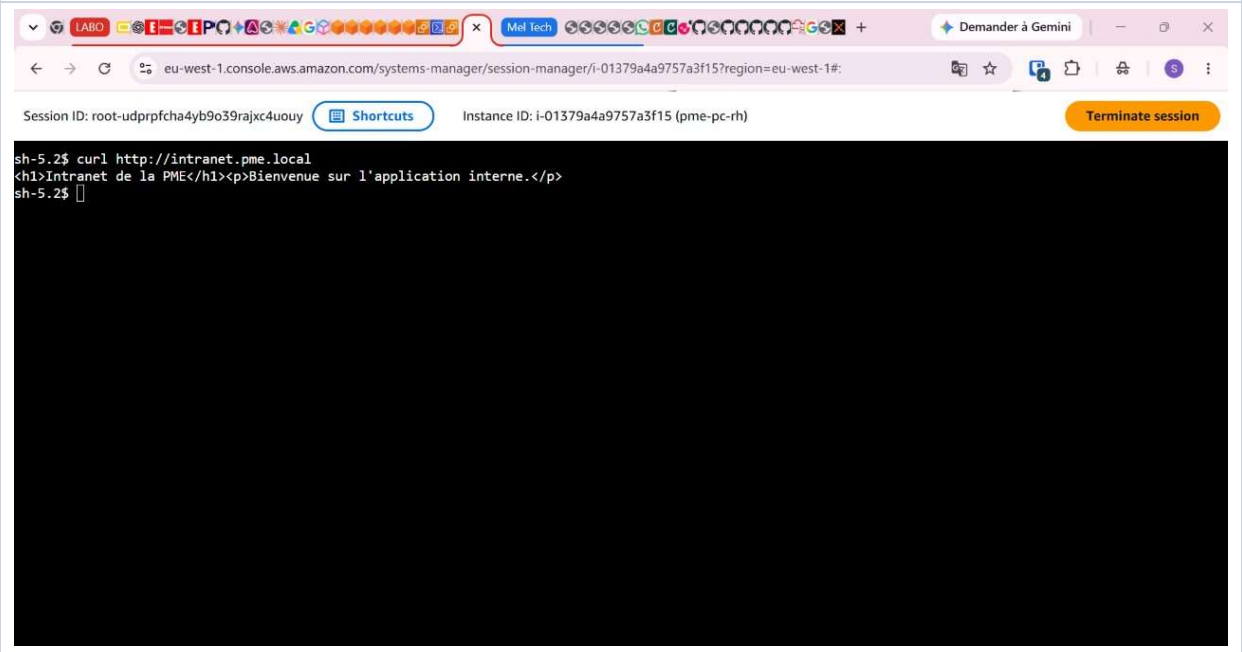


Figure 9 *Accès au serveur applicatif depuis un poste : commande curl et code HTML renvoyé.*

7.7. Le Windows Server

La connexion au serveur `serveur-ad-fichiers` s'effectue en Bureau à distance via AWS Systems Manager (Fleet Manager → Remote Desktop), sans exposer le port RDP sur Internet. Le Gestionnaire de serveur confirme l'installation des deux rôles : **AD DS** et **File and Storage Services**.

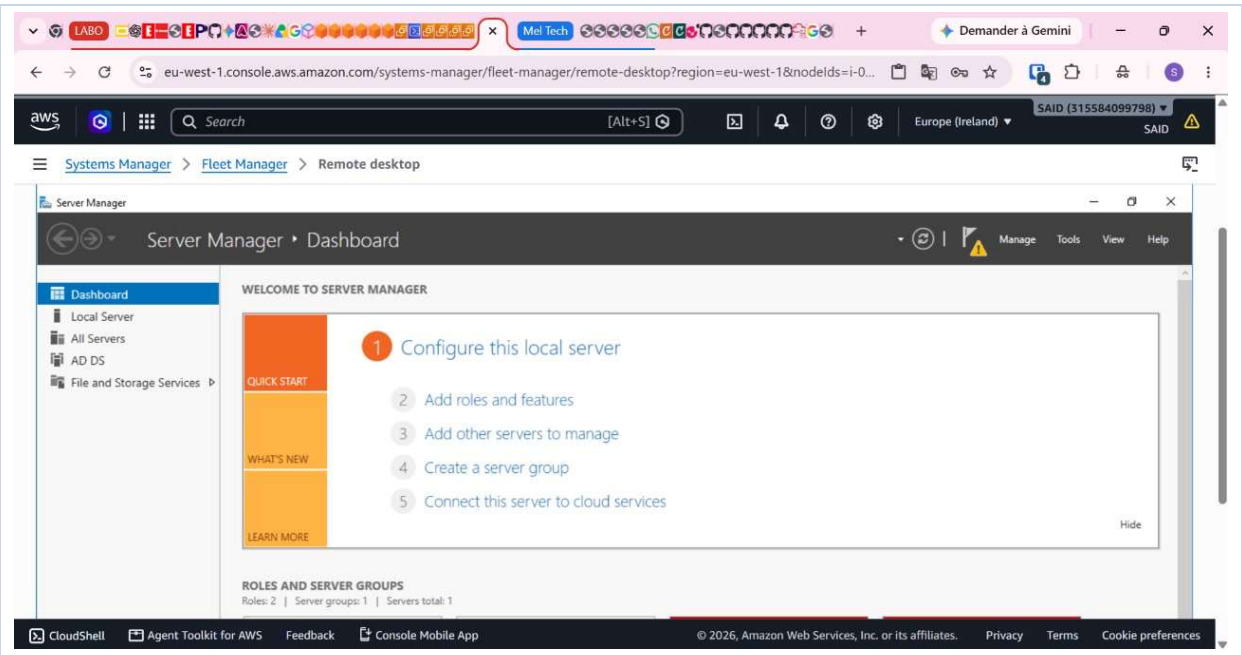


Figure 10 *Gestionnaire de serveur Windows — rôles AD DS et File and Storage Services installés.*

Le dossier de partage créé par le script `user_data` est ensuite vérifié dans l'Explorateur de fichiers, à la racine du disque `C:`.

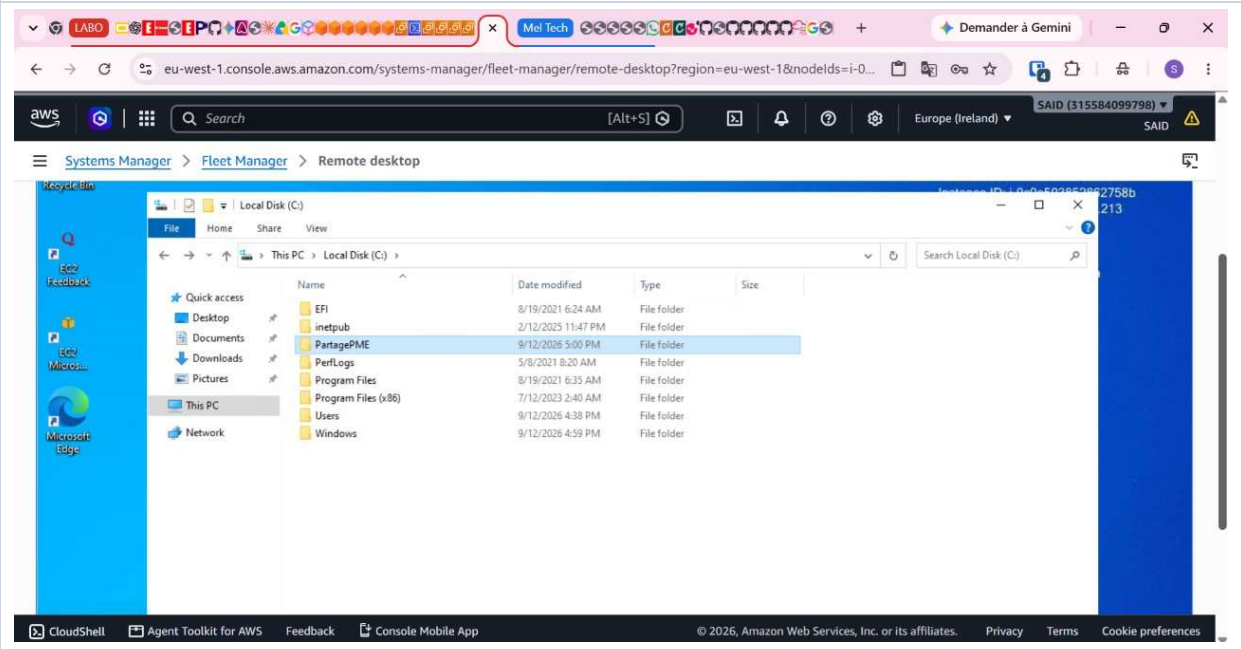


Figure 11 *Explorateur de fichiers — dossier de partage PartagePME.*

08 Analyse, Limites et Perspectives

8.1. Choix de conception

- **Instance NAT plutôt que NAT Gateway** : économie significative pour une petite structure, au prix d'une moindre résilience.
- **for_each pour les postes** : provisioning scalable, un nouveau poste ne nécessitant qu'une entrée de liste.
- **DNS interne Route 53** : noms lisibles au lieu d'adresses IP, indépendance vis-à-vis des changements d'adressage.
- **Variables sensibles et tags communs** : bonnes pratiques de gestion des secrets et d'inventaire.

8.2. Limites actuelles

- Les postes peuvent atteindre les serveurs sans restriction fine ; aucune segmentation par utilisateur ou par rôle.
- Le serveur Windows n'est pas promu contrôleur de domaine ; l'annuaire Active Directory n'est pas peuplé.
- Droits IAM étendus (`AdministratorAccess`), état Terraform local et mot de passe fourni en clair dans un fichier de variables : acceptables en laboratoire, à proscrire en production.
- Instance NAT unique : point de défaillance unique pour l'accès Internet sortant.

8.3. Perspectives

Cette infrastructure sert de base au projet suivant (approche Zero Trust) : restriction fine des flux entre segments, chiffrement, journalisation et moindre privilège. La comparaison « avant / après » sur cette même base illustre une démarche réaliste : construire une infrastructure, puis la sécuriser.

09 Nettoyage des Ressources

```
terraform destroy   # confirmer avec "yes"
```

Après l'exécution, il convient de vérifier dans la console EC2 que toutes les instances, et en particulier le Windows Server (ressource la plus coûteuse), sont à l'état « Terminated ».

10 Synthèse des Compétences Validées

Domaine	Compétences techniques validées
Architecture réseau cloud	Conception d'une architecture segmentée (DMZ / Serveurs / Postes) : VPC, sous-réseaux, tables de routage, Internet Gateway.
Routage AWS	Routes publiques et privées, instance NAT (`source_dest_check`, IP forwarding, masquage).
Infrastructure as Code	Provisioning déclaratif Terraform : providers, variables sensibles, locals, `for_each`, outputs, cycle `init/plan/apply/destroy`.
Administration Windows	Déploiement d'un Windows Server (rôles AD DS et Fichiers) via script PowerShell `user_data`.
Filtrage réseau	Security Groups par segment, flux limités aux ports et plages nécessaires.
Services d'infrastructure	DNS interne (Route 53 Private Hosted Zone), serveur intranet, site web public.
Maîtrise des coûts	Budget AWS avec alertes, choix NAT Instance, nettoyage via `destroy`.

Ce projet démontre la capacité à reconstruire, de manière reproductible et documentée, l'équivalent cloud d'un réseau de PME, et constitue le socle d'un travail ultérieur de sécurisation.

Who Is

SAID